Изброени типове: data Color = Red | Green | Blue ||| colorToString :: Color -> String ||| colorToString Red = "Червено"
data Shape = Circle Double | **Triangle** Double Double Double ||| **type** Name = String ||| **newtype** Meters = Meters Double
data Student = Student { **studentName** :: String} ||| **deriving** (Show, Eq, Ord, Enum, Bounded) ||| **instance** className **where**
Бинарни дървета: data Tree a = Empty | Node a (Tree a) (Tree a) deriving (Show, Eq) ||| **Двоично наредено дърво (BST)** |||
Абстрактни синтактични д. (AST) ||| **>>=** за списъци е **concatMap** ||| **splitAt** elNum list ||| import Data.Char (isDigit, isUpper)
Обхождания: inorder (LVR), preorder (VLR), postorder (LRV), Level order (BFS) ||| **case** something((x,xs) for lists) **of** ||| Nothing **->** …
>>= (bind), >> (then), return - return x = Just x ||| **do**-нотацията е синтактична захар за верига от >>= ||| **Правила** за do:
1. x <- action - изпълнява action и именува резултата x , **2**. action без <- - изпълнява action, игнорира резултата (като >>)
3. Последният ред е резултатът от целия do блок **4**. let x = expr - обикновен let (без in!) ||| main :: IO () main = do |||
x <- action - изпълнява IO действие, именува резултата, let x = expr - обикновено свързване (без IO), return x - обвива чиста стойност в IO (НЕ спира изпълнението!)

· data Maybe a = Nothing | Just a - Data.Maybe (catMaybe – Премахва Nothing стойностите и разопакова Just, isJust, isNothing, fromMaybe)
· data Either a b = Left a | Right b (a – Error) ||| **lookup** something [(l,r)]

```
treeSize :: Tree a -> Int
treeSize Empty = 0
treeSize (Node _ l r)  = 1 + treeS l + treeS r
```

```
treeHeight :: Tree a -> Int
treeHeight Empty = 0
tH (Node _ l r) = 1 + max (tH l) (tH r)
```

```
(>>) :: Maybe a -> Maybe b -> Maybe b
Nothing >> _ = Nothing
Just _  >> m = m
```

```
treeElem :: Eq a => a -> Tree a -> Bool
treeElem _ Empty = False
treeElem x (Node v l r) = x == v || treeElem
x l || treeElem x r
```

```
treeLevel :: Int -> Tree a -> [a]
treeL _ Empty= [] treeL 0(Node v _ _)= [v]
treeLevel n (Node _ l r) | n > 0 = treeL (n-
1) l ++ treeL (n-1) r       | otherwise = []
```

```
inorder :: Tree a -> [a]  == toList
inorder Empty  = []
inorder (Node v l r)   = inorder l ++ [v] ++
inorder r // the others -> changing the order
```

```
levelOrder :: Tree a -> [a]
levelOrder tree = go [tree]
  where
  go [] = []
  go (Empty : rest)  = go rest
  go (Node x l r : rest) = x : go (rest ++ [l, r])
```

```
bstSearch :: Ord a => a -> Tree a -> Bool
bstSearch _ Empty = False
bstSearch x (Node v l r)
   | x == v  = True
   | x < v = bstSearch x l
   | otherwise = bstSearch x r
```

```
treeMap :: (a -> b) -> Tree a -> Tree b
treeMap _ Empty = Empty
tM f (Node x l r)= Node (tM f l) (f x) (tM f r)
treeFoldr :: (a -> b -> b) -> b -> Tree a -> b
treeFoldr _ acc Empty = acc
tF f acc (Node l x r) = tF f (f x (tF f acc r)) l
```

```
bstInsert :: Ord a => a -> Tree a -> Tree a
bstInsert x Empty = Node x Empty Empty
bstInsert x (Node root l r)
    | x < root = Node root (bstInsert x l) r
    | x > root = Node root l (bstInsert x r)
    | otherwise = Node root l r

fromList :: Ord a => [a] -> Tree a
fromList = foldl (flip bstInsert) Empty

treePaths :: Tree a -> [[a]]
treePaths Empty = []
treePaths (Node r Empty Empty) = [[r]]
treePaths (Node root l r) =
   map (root:) (treePaths l) ++ map (root:)
(treePaths r)
```

```
bstDelete :: Ord a => a -> Tree a -> Tree a
bstDelete _ Empty = Empty
bstDelete x (Node root l r)
    | x < root = Node root (bstDelete x l) r
    | x > root = Node root l (bstDelete x r)
    | otherwise = deleteNode (Node root l r)
where
deleteNode (Node _ Empty Empty) = Empty
deleteNode (Node _ l Empty) = l
deleteNode (Node _ Empty r) = r
deleteNode (Node _ l r) =
   let m = findMin r
   in Node m l (bstDelete m r)

findMin :: Tree a -> a
findMin (Node root Empty _) = root
 findMin (Node _ l _) = findMin l
```

```
// Когато не ни трябва резултатът от
предишната стъпка:
andThen :: Maybe a -> (a -> Maybe b) ->
Maybe b                 ==      >>= (bind)
andThen Nothing = Nothing
andThen (Just x) f = f x      // same for Either
 process :: [(String, String)] -> Maybe Int
 process env = lookup' "x" env >>= safeRead
 >>= validate
   where
validate n = if n > 0 then Just n else Nothing
rotateLeft :: IntTree -> IntTree
rotateLeft (Node x leftA (Node y leftB
rightC)) =Node y (Node x leftA leftB) rightC
rotateLeft tree = tree
rotateRight (Node x (Node y leftA rightB)
rightC) =Node y leftA(Node x rightB rightC)
```

```
main :: IO ()       -- програма без резултат
getLine :: IO String      -- чете ред от stdin
putStrLn::String->IO()--печатареднастdout
getLine  :: IO String              -- чете ред
getChar  :: IO Char              -- чете символ
readLn   :: Read a => IO a  -- чете и парсва
getContents :: IO String  -- чете целия stdin
putStr :: String ->IO()-- печата без нов ред
putStrLn  :: String->IO()-- печата с нов ред
print::Show a => a ->IO() -- show +putStrLn
putChar   :: Char -> IO ()  -- печата символ
```

```
readFile :: FilePath -> IO String -- Четене
writeFile :: FilePath -> String -> IO () --
Писане (презаписва!)
appendFile :: FilePath -> String -> IO ()
lines, words, <- from prelude
process and read from file
processFile :: FilePath -> IO ()
processFile path = do
  content <- readFile path
  let linesList = lines content
  let numbered = zipWith (\i l -> show i ++
": " ++ l) [1..] linesList
  mapM_ putStrLn numbered
```

```
import Control.Monad (when, forM_)
mapM_   :: (a -> IO b) -> [a] -> IO ()     --
map + изпълняване, без резултат
mapM    :: (a -> IO b) -> [a] -> IO [b]    --
map + изпълняване, със резултат
forM_   :: [a] -> (a -> IO b) -> IO ()     --
обратен ред на аргументите
forM    :: [a] -> (a -> IO b) -> IO [b]
sequence_ :: [IO a] -> IO ()               --
изпълнява поредица
when     :: Bool -> IO () -> IO ()         --
условно изпълнение
unless   :: Bool -> IO () -> IO ()
```

```
parseCSV :: String -> [[String]]
parseCSV = map (splitOn ',') . lines

readCSV :: FilePath -> IO [[String]]
readCSV path = do
  content <- readFile path
  return (parseCSV content)
```

```
splitOn :: Char -> String -> [String]
splitOn _ [] = [""]
splitOn sep (c:cs)
  | c == sep  = "" : splitOn sep cs
  | otherwise = let (first:rest) =splitOn sep cs
                in (c:first) : rest
```

```
writeCSV :: FilePath -> [[String]] -> IO ()
writeCSV path rows = writeFile path
csvContent      where
csvConten=unlines(map(intercalate ",")rows)
intercalate sep xs=concat (intersperse sep xs)
intersperse _ []     = [] intersperse _ [x]   = [x]
   intersperse sep (x:xs) = x : sep : intersperse
sep xs
```